

PROGRAMACIÓN EN RADIO DEFINIDA POR SOFTWARE (GNURADIO)

Sebastian Felipe Castellanos Garnica - 2225187

Brayan David Castañeda Zarate - 2225188

Juan Diego Gomez Rodriguez - 2225218

https://github.com/dieguinho0003/2026_2_CommII_A1_GA1J.git

Escuela de Ingenierías Eléctrica, Electrónica y de Telecomunicaciones

Universidad Industrial de Santander

1 de septiembre de 2026

Abstract—This laboratory session involved the study, analysis, and implementation of GNU Radio blocks with distinct functions—specifically an accumulator, a differentiator, and a statistical block. Various signals were fed into these blocks to observe their output behavior; these signals were generated using specific input vectors. The blocks' response times and their ability to reflect the input mean were analyzed. Once the blocks were connected and verified to be functioning correctly, they were uploaded to a GitHub repository to document the work performed. This process also entailed understanding the Git environment, learning the necessary commands to upload files, and validating the procedure.

Index Terms—GNU Radio, Python blocks, Software-Defined Radio (SDR), Accumulator, Differentiator, Statistical processing.

I Introducción

Cuando se trabaja en actividades académicas o profesionales que requieren colaboración en equipo, resulta clave contar con herramientas que permitan organizar el trabajo, compartir avances y hacer seguimiento del aporte de cada integrante de forma práctica. Por eso, en este laboratorio se utilizó GitHub, una plataforma en la nube basada en el sistema de control de versiones Git, que permite almacenar y colaborar en proyectos de código fuente [1]. Gracias a esta herramienta fue posible centralizar todos los archivos desarrollados por el equipo y facilitar el trabajo conjunto entre los integrantes.

Para el desarrollo del laboratorio se empleó también GNU Radio, una plataforma de código abierto pensada para el diseño de sistemas de comunicación y procesamiento de señales a partir de bloques funcionales conectados en un diagrama de flujo. Una de sus ventajas es que permite integrar código en Python para crear bloques personalizados según las necesidades del siste-

ma que se esté diseñando [2]. Con esta herramienta se implementaron tres bloques propios: un acumulador, un diferenciador y un bloque estadístico, cuyo objetivo era analizar cómo se comporta una señal al pasar por cada uno de ellos.

El bloque acumulador se encarga de sumar de manera continua los valores de la señal de entrada, entregando a la salida un resultado que depende de todo lo acumulado hasta ese momento. El diferenciador, por su parte, realiza la operación de diferenciación sobre la señal, de modo que su salida refleja qué tan rápido está cambiando dicha señal en el tiempo. Finalmente, el bloque estadístico permite calcular distintos parámetros a partir de un vector de valores, como la media, la varianza y la media cuadrática, entre otros. Este tipo de operaciones son ampliamente utilizadas dentro del procesamiento digital de señales [3]. Una vez obtenidas las señales de salida, se analizaron para verificar el correcto funcionamiento de los bloques. Finalmente, los archivos de GNU Radio se subieron a GitHub para facilitar el trabajo colaborativo y organizar los resultados del laboratorio.

I-A Bloque Acumulador

Este bloque se configuró para realizar una suma secuencial de las muestras de la señal de entrada. Dentro del Python Block se implementó una función orientada a la acumulación iterativa de los datos. La suma acumulativa se asigna directamente a la salida, permitiendo observar el comportamiento integrador de la señal a lo largo de las muestras.

Para verificar el correcto funcionamiento, se diseñó el esquema de bloques ilustrado en la Figura 1. Inicialmente, se emplea un Vector Source como señal de entrada dirigida hacia el bloque acumulador; dicha

señal se regula mediante un bloque Throttle a una tasa de muestreo de 32k, lo que permite visualizar y comparar la señal original con aquella que atravesó el bloque de acumulación.

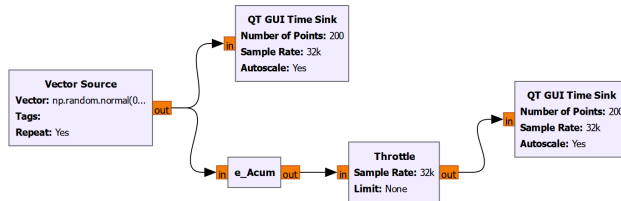


Figura 1: Diagrama de Bloques en GNU Radio del bloque acumulador

Para lograr el comportamiento deseado dentro del bloque personalizado `e_Acum`, se programó el siguiente algoritmo en lenguaje Python. En este código se define una variable de estado `self.acum` que almacena la suma acumulada entre iteraciones y procesa muestra a muestra el vector de entrada:

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):
    def __init__(self):
        gr.sync_block.__init__(
            self,
            name='e_Acum',
            in_sig=[np.float32],
            out_sig=[np.float32]
        )
        self.acum = 0.0

    def work(self, input_items, output_items):
        in0 = input_items[0]
        out = output_items[0]

        for i in range(len(in0)):
            self.acum += in0[i]
            out[i] = self.acum

        return len(output_items[0])
```

I-B Bloque Diferenciador

El bloque diferenciador `e_Diff` Figura 2 se diseñó con memoria entre iteraciones: calcula la diferencia respecto al valor anterior, actualiza dicho estado y entrega el resultado al flujo de salida. Su estructura general es similar a la del bloque acumulador.

Para derivar en tiempo discreto con el bloque `e_Diff`, se programó un algoritmo en Python que usa la variable `self.muestra_anterior` para guardar el último valor procesado y calcular continuamente las diferencias finitas entre iteraciones:

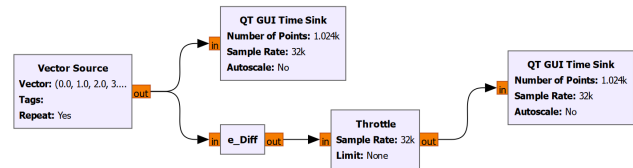


Figura 2: Diagrama de Bloques en GNU Radio del bloque diferenciador

```
import numpy as np
from gnuradio import gr

class blk(gr.sync_block):
    def __init__(self):
        gr.sync_block.__init__(
            self,
            name='e_Diff',
            in_sig=[np.float32],
            out_sig=[np.float32]
        )
        self.muestra_anterior = 0

    def work(self, input_items, output_items):
        x = input_items[0]
        y0 = output_items[0]

        # Calcula la diferencia punto a punto en el bloque actual
        y0[0] = x[0] - self.muestra_anterior
        y0[1:] = np.diff(x)

        self.muestra_anterior = x[-1]

        return len(y0)
```

I-C Bloque Estadístico

Aquí se implementó un bloque síncrono con cinco salidas independientes para el cálculo simultáneo de métricas en tiempo real: media, varianza, desviación estándar, valor RMS y potencia promedio. Su diagrama esquemático conectó una fuente vectorial de prueba directamente al bloque como se ilustra en la Figura 3, visualizando cada una de las métricas a través de bloques QT GUI Number Sink.

Para llevar a cabo el procesamiento de métricas en tiempo real sobre el vector de entrada, se desarrolló el algoritmo en Python implementado en el bloque personalizado `Bloque_Estadístico`. Este código configura un bloque síncrono provisto de cinco canales de salida independientes, sobre los cuales se asignan los resultados de los cálculos vectoriales de media, varianza, desviación estándar, valor máximo y valor eficaz (RMS) obtenidos mediante la librería `numpy`.

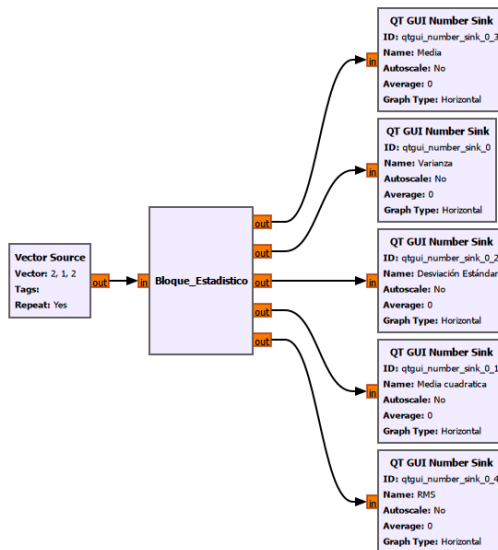


Figura 3: Diagrama de Bloques en GNU Radio del bloque estadístico

II RESULTADOS Y ANÁLISIS DE DATOS

II-A Bloque Acumulador

Para el análisis del bloque acumulador `e_Acum`, se utilizó como señal de entrada una secuencia de números aleatorios con distribución normal gaussiana de media cero ($\mu = 0$) generada mediante el comando `np.random.normal(0, 1, 1000)` en el bloque `Vector Source`. La señal de entrada y la salida del acumulador se visualizan en la Figura 4.

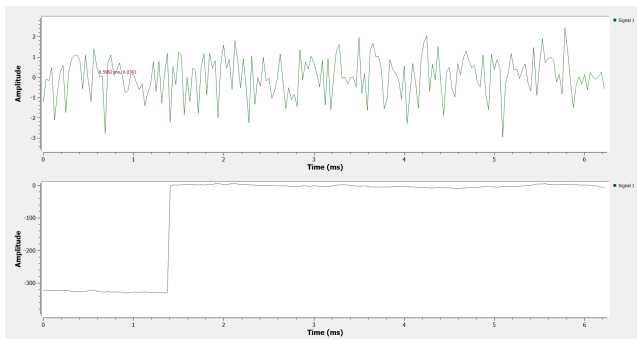


Figura 4: Señal de entrada Gaussiana (arriba) y salida del bloque acumulador (abajo).

El comportamiento discreto del acumulador está en funcionamiento por la ecuación en diferencias:

$$y[n] = y[n - 1] + x[n]$$

donde $x[n]$ representa la muestra de la señal de entrada en el instante n , y $y[n]$ es el valor acumulado en la salida.

- **Señal de entrada $x[n]$:** Fluctuación aleatoria simétrica alrededor de cero, con amplitudes entre -4 y 4 .
- **Efecto de acumulación $y[n]$:** La suma continua actúa como integrador discreto, generando una caminata aleatoria (*random walk*).
- **Dinámica temporal:** En la ventana de 0 a 6,2 ms del *QT GUI Time Sink*, la señal exhibe un comportamiento no estacionario. Al retener memoria infinita del proceso, la salida evoluciona desde valores fuertemente negativos (inferiores a -300) hacia la zona cercana a cero, reflejando cómo pequeños desbalances en la entrada acumulan y desplazan progresivamente la señal.

II-B Bloque Diferenciador

La señal usada fue una señal rampa que oscila entre 0 y 20 unidades de amplitud., la cual fue realizada mediante el vector (0,2,4,6,8,10,12,14,16,18,20),

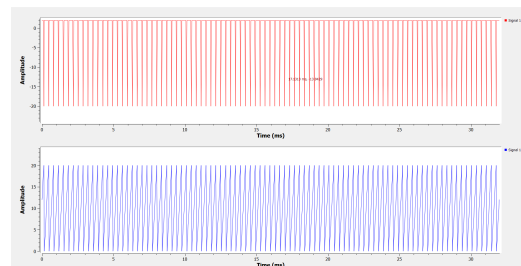


Figura 5: Señal de entrada y salida para el bloque diferenciador

Se observa que la señal de entrada (señal azul) crece de forma lineal de 0 a 20 en un intervalo de tiempo continuo y presenta una caída abrupta desde 20 de regreso a 0.

El comportamiento matemático del diferenciador se define como:

$$y(t) = \frac{dx(t)}{dt} \quad (1)$$

- **Tramo Ascendente (Pendiente Constante y Positiva):**

La tasa de cambio $\frac{\Delta x}{\Delta t}$ es constante:

$$\frac{dx(t)}{dt} \approx \frac{20 - 0}{\Delta t} > 0 \quad (2)$$

Esto genera un valor constante positivo cercano a 2 en la señal de salida.

■ Discontinuidad:

Cuando la señal cae de 20 a 0 en un tiempo extremadamente pequeño ($\Delta t \rightarrow 0$), la derivada es fuertemente negativa:

$$\frac{dx(t)}{dt} \approx \frac{0 - 20}{\Delta t} \ll 0 \quad (3)$$

Esto produce los impulsos o picos de valor negativo que como se pueden ver en la gráfica alcanzan -20 exactamente en cada ciclo del flanco de bajada.

Se realizó una prueba con una señal senoidal, la cual entregó a la salida

II-C Bloque Estadístico

Para validar el funcionamiento del bloque personalizado `Bloque_Estadístico`, se ingresó como señal de entrada el vector periódico y se realizaron los cálculos: *QT GUI Number Sink* de la Figura 6.

$$x = (2, 1, 2)$$

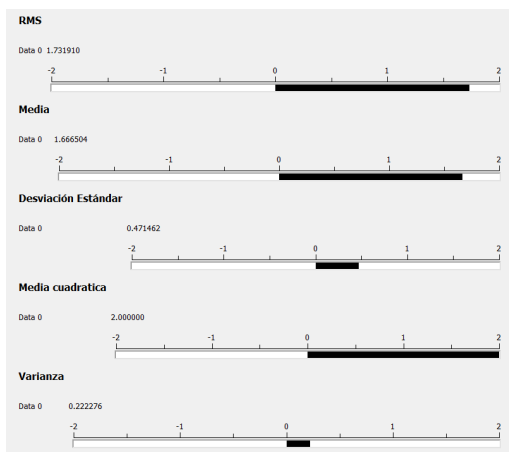


Figura 6: Resultados de los parámetros estadísticos para la señal de entrada.

- **Media (μ):** Promedio aritmético de las muestras del vector.

$$\mu = \frac{2 + 1 + 2}{3} = \frac{5}{3} \approx 1,666667$$

En el visor se obtuvo un valor medido de 1,666626, lo cual concuerda con el cálculo teórico.

- **Varianza (σ^2):** Promedio de las desviaciones al cuadrado respecto a la media.

$$\sigma^2 = \frac{2}{9} \approx 0,222222$$

- **Desviación Estándar (σ):** Valor teórico $\approx 0,471404$, medido 0,471419 ✓
- **Media Cuadrática ($E[x^2]$):** Valor teórico 3,000000; sin embargo, la salida `out_máx` mostró 2,0, correspondiente al máximo del vector y no a la media cuadrática (error de etiquetado en la GUI).
- **Valor RMS (x_{rms}):** Valor teórico $\approx 1,7320508$, medido 1,732016 ✓

III Conclusiones

Se confirmó que el bloque acumulador funciona sumando continuamente todo lo que le llega. Aunque la señal de entrada tenía un promedio de cero, la salida no se quedó quieta en cero, sino que fue subiendo y bajando de forma aleatoria porque el bloque guarda el historial de cada dato y cualquier pequeño cambio afecta el resultado total con el tiempo.

En el bloque diferenciador se comprobó que este bloque sirve para detectar qué tan rápido cambia una señal. Al probarlo con la señal en forma de rampa, la salida mostró un valor constante mientras la señal subía suavemente, pero dio un pico negativo bastante alto justo en el momento en que la señal caía de golpe a cero.

Para el bloque estadístico se comprobó que es capaz de calcular varias características de la señal al mismo tiempo y mostrarlas en salidas independientes. Al probarlo con el vector de datos, los valores obtenidos en las pantallas coincidieron con los cálculos matemáticos, demostrando que el bloque procesa y actualiza correctamente la información en tiempo real.

Referencias

- [1] @BOOK11244343, author=Ayodele, Ayodeji and Burtally, Irshad and Condah, Ali, booktitle=GitHub Foundations Certification Guide: Essential skills, real-world labs, and exam strategies for GitHub beginners, year=2025,
- [2] E. Blossom, "GNU Radio: Tools for exploring the radio frequency spectrum," IEEE Communications Magazine, vol. 42, no. 8, pp. 84–89, 2004.
- [3] S. K. Mitra, "Digital signal processing: A computer-based approach to discrete-time systems," IEEE Signal Processing Magazine, vol. 18, no. 6, pp. 14–20, 2001.